

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Нейроінформатика»

Робота №3

Тема «Нейромережі з латеральними зв'язками»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Афанасьєва А.В.

2025

МЕТА

Вивчити моделі нейронних мереж з латеральними зв'язками та методи їхнього навчання; порівняти можливості нейромереж з латеральними зв'язками з можливостями нейромереж прямого поширення та можливостями мереж зі зворотними зв'язками; ознайомитися з програмними засобами, що моделюють нейромережі з латеральними зв'язками.

ЗАВДАННЯ

Завдання на роботу:

1. Ознайомитися з конспектом лекцій та рекомендованою літературою.
2. Вивчити архітектуру нейромереж з латеральними зв'язками, а також методи їхнього навчання.
3. Використовуючи документацію рекомендованого викладачем програмного засобу, вивчити його архітектуру і компоненти, призначені для моделювання і навчання нейромереж з латеральними зв'язками, введення і виведення даних, підготовки звітів (виведення і відображення результатів роботи).
4. Визначити за таблицею 1.1 згідно з номером варіанта студента (обирається за журналом) номер підрозділу Додатку В, що містить опис прикладної задачі, та номери контрольних питань до даної роботи, відповіді на які належить навести у звіті.

Задача: В.2; Номер контрольних питань: 3, 13, 29, 33, 49

Б.2 Прогнозування надійності виробів електронної техніки

Вироби електронної техніки (ОСТ 11 ОДО.052.005), далі ВЕТ, являють собою ненадійні вироби, що можуть бути охарактеризовані наступними параметрами:

- x1 – струм сітки першої зворотний, мкА;
- x2 – струм сітки першої зворотний при перегарті, мкА;
- x3 – струм анода, мА;
- x4 – струм сітки другий, мА;
- x5 – крутість характеристики, мА/В;
- x6 – крутість характеристики при недожару, мА/В;
- x7 – струм розжарення, мА;
- x8 – ємність вхідна, пф;
- x9 – ємність вихідна, пф;
- x10 – ємність прохідна, пф;
- x11 – час готовності, с;
- x12 – коефіцієнт підсилення в тріодному включенні;
- x13 – напруга контактної різниці потенціалів, В;
- x14 – відносна зміна струму анода, %;
- x15 – струм емісії при недожару, А;
- x16 – опір оксидного шару, Ом;
- x17 – струм емісії, А;
- x18 – відносна зміна крутості характеристики після іспитів на довговічність 5000 год., %.

Будучи піддані навантаженню в процесі експлуатації ці вироби можуть виходити з ладу в різний час. Задачею контролю якості цих виробів при завершенні виробництва, а також в експлуатації є прогнозування ресурсу (терміну служби) ВЕТ. Причому на практиці

цілком достатньо якісного прогнозу, тобто досить спрогнозувати, чи зможе конкретний виріб прослужити заданий термін чи ні. Для побудови моделі якісної залежності між параметрами і класом ВЕТ можна використовувати експериментально отримані і пронормовані дані.

Формалізувавши задачу маємо: 18 вхідних ознак, що містять параметри ВЕТ 1 цільову ознаку - номер класу ВЕТ (1 - дефектний, 0 - гідний).

Кількість шарів: 2, кількість нейронів у шарах: 8-1, назва функції активації: логістична сигмоїдна.

5. Використовуючи Python для відповідного варіанту студента вхідних даних вирішити прикладну задачу на основі нейромереж Кохонена LVQ, при цьому окремо дослідити властивості різних методів навчання. Для тієї ж задачі сформувати кластери на основі карт ознак, що самоорганізуються (SOM).

6. Зберегти у файлі на диску початкові параметри нейромоделей, результати їхнього навчання (матрицю ваг та структуру з значенням функцій активації і кількості нейроелементів у шарах) та робити для навчальної та контрольної вибірок (помилку класифікації (оцінювання), значення на входах і виході, час навчання, час класифікації).

7. Результати виконання пунктів 5-6 занести в таблицю, стовпці якої повинні мати назви: назва архітектури нейромережі, кількість шарів, кількість нейронів у шарах, функції активації нейронів у шарах, метод навчання, час навчання, час класифікації навченої мережі для навчальної вибірки, помилка класифікації для навчальної вибірки, час класифікації навченої мережі для тестової вибірки, помилка класифікації для тестової вибірки.

8. Проаналізувати отримані результати і дати порівняльну характеристику мереж з латеральними зв'язками SOFM та LVQ. Дати порівняльну характеристику мереж з латеральними зв'язками, мереж зі

зворотними зв'язками та мереж прямого поширення за швидкістю навчання і точністю класифікації; принципами побудови архітектури.

9. Відповісти на контрольні питання.

10. Оформити звіт з роботи.

ВИКОНАННЯ

1 SOFM

Код

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from minisom import MiniSom
import numpy as np
import json
import os

# Генерація даних
N = 500

X = np.random.rand(N, 18)
y = np.random.randint(0, 2, N)

# Розділення на навчальну та тестову вибірки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.8, random_state=42
)

# --- Навчання SOFM ---
# Створюємо карту 8x8 нейронів
map_size = (8, 8)
som = MiniSom(
    x=map_size[0],
    y=map_size[1],
    input_len=18,
    sigma=1.0, # радіус сусідства
    learning_rate=0.5,
    random_seed=42,
)

# Ініціалізація ваг
som.random_weights_init(X_train)

# --- Маркування нейронів ---
# Кожен нейрон отримує мітку за більшістю зразків, які до нього потрапили
neuron_labels = np.zeros(map_size)
neuron_counts = np.zeros(map_size)
```

```

for i, sample in enumerate(X_train):
    winner = som.winner(sample) # координати нейрона-переможця
    neuron_labels[winner] += y_train[i]
    neuron_counts[winner] += 1

# Перетворюємо в мітку класу (0 або 1) на основі більшості
for i in range(map_size[0]):
    for j in range(map_size[1]):
        if neuron_counts[i, j] > 0:
            # Якщо більшість зразків класу 1
            if neuron_labels[i, j] / neuron_counts[i, j] >= 0.5:
                neuron_labels[i, j] = 1
            else:
                neuron_labels[i, j] = 0
        else:
            neuron_labels[i, j] = -1 # нейрон без зразків

print("\nКарта міток нейронів (8x8):")
print(neuron_labels.astype(int))

# --- Класифікація ---
def classify(sample):
    """Класифікує новий зразок"""
    winner = som.winner(sample)
    return int(neuron_labels[winner])

# Передбачення на тренувальних даних
train_pred = [classify(sample) for sample in X_train]
train_acc = accuracy_score(y_train, train_pred)

# Передбачення на тестових даних
test_pred = [classify(sample) for sample in X_test]
test_acc = accuracy_score(y_test, test_pred)

print(f"\nТочність на тренувальних даних: {train_acc * 100:.2f}%")
print(f"Точність на тестових даних: {test_acc * 100:.2f}%")

# --- Візуалізація ---
# Візуалізація карти нейронів з кольорами класів
plt.figure(figsize=(10, 8))
plt.imshow(neuron_labels, cmap="RdBu", origin="lower", vmin=-1, vmax=1)
plt.colorbar(label="Клас (0=синій, 1=червоний)")
plt.title("Карта SOFM з маркуванням нейронів")
plt.xlabel("X координата")
plt.ylabel("Y координата")

# Додаємо текстові мітки
for i in range(map_size[0]):
    for j in range(map_size[1]):
        if neuron_counts[i, j] > 0:
            color = "white" if neuron_labels[i, j] == 1 else "black"
            plt.text(
                j,
                i,
                str(int(neuron_labels[i, j])),
                ha="center",

```

```

        va="center",
        color=color,
        fontsize=8,
    )
    else:
        plt.text(j, i, "?", ha="center", va="center", color="gray",
        fontsize=8)
plt.show()

data = {"Train Accuracy": train_acc, "Test Accuracy": test_acc}
current_folder = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(current_folder, "sofm_output.json")
with open(file_path, encoding="utf-8", mode="w") as f:
    json.dump(data, f, indent=2)

```

Результати на графіку

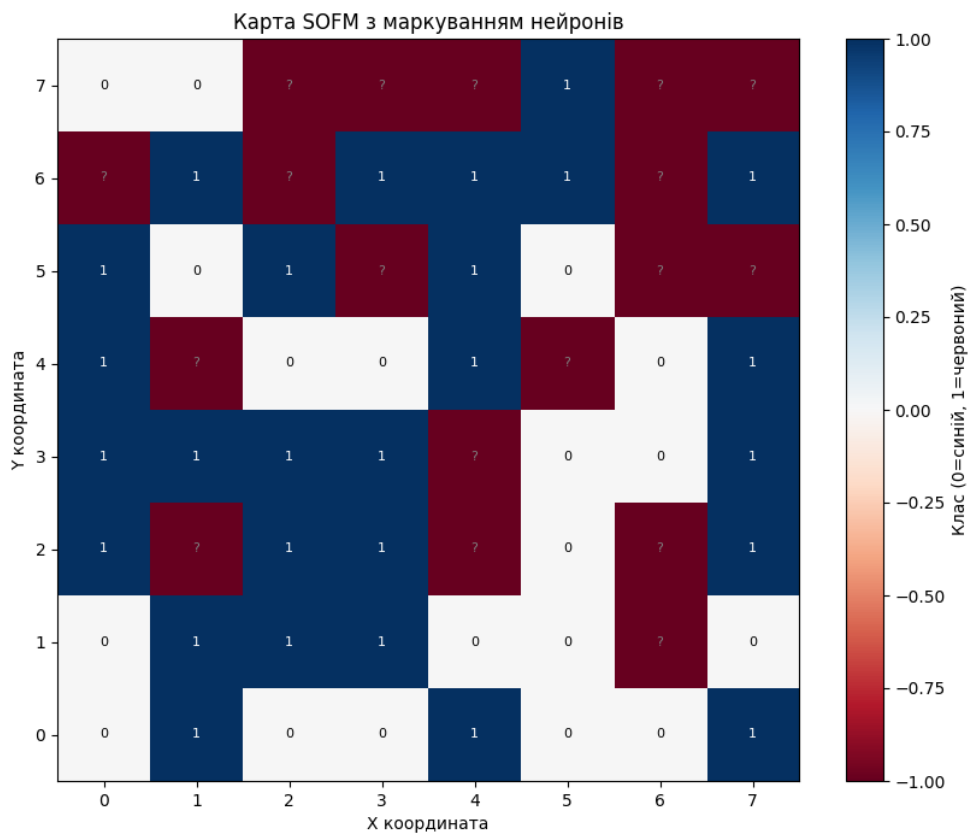


Рисунок 1.1 — Результати виконання

Результати в консолі

Карта міток нейронів (8×8):
[[0 1 0 0 0 0 0 0]
[1 0 1 1 0 1 -1 1]
[1 -1 1 0 -1 0 -1 1]
[1 0 0 0 -1 1 1 1]
[1 -1 0 1 0 -1 1 0]
[0 0 0 -1 0 0 -1 -1]
[-1 1 -1 1 1 0 -1 1]
[1 0 -1 -1 -1 1 -1 -1]]

Точність на тренувальних даних: 74.00%
Точність на тестових даних: 47.75%

Результати у файлі

```
{  
  "Train Accuracy": 0.78,  
  "Test Accuracy": 0.5125  
}
```

Порівняння результатів

Під час виконання було проведено кілька експериментів з різними вхідними даними. Деталі нижче:

Розмір карти	Кількість зразків у вибірці	Розбиття вибірки на тестову та навчальну	Точність на тренувальних даних, %	Точність на тестових даних, %
4 на 4	500	80 на 20	65	51.75
5 на 5	500	80 на 20	66	46.50
6 на 6	500	80 на 20	73	48.25
7 на 7	500	80 на 20	71	46
8 на 8	500	80 на 20	79	52
		70 на 30	72.67	53.14
	1000	80 на 20	69	50.62
	5000	80 на 20	57.30	51.38

2 LVQ

Kod

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np
import json
import os

class LVQ:
    """
    Learning Vector Quantization (LVQ1) - ручна реалізація
    """

    def __init__(self, n_prototypes_per_class=3, learning_rate=0.1,
max_epochs=100):
        """
        Параметри:
        - n_prototypes_per_class: кількість прототипів на клас
        - learning_rate: швидкість навчання ( $\alpha$ )
        - max_epochs: максимальна кількість епох
        """

        self.n_prototypes_per_class = n_prototypes_per_class
        self.learning_rate = learning_rate
        self.max_epochs = max_epochs
        self.prototypes = [] # список прототипів (вектори ваг)
        self.prototype_labels = [] # мітки класів прототипів

    def fit(self, X, y):
        """
        Навчання LVQ
        X: вхідні дані (n_samples, n_features)
        y: цільові мітки (0 або 1)
        """

        n_features = X.shape[1]
        self.prototypes = []
        self.prototype_labels = []

        # Ініціалізація прототипів: випадкові зразки з кожного класу
        for class_label in [0, 1]:
            X_class = X[y == class_label]
            n_prototypes = min(self.n_prototypes_per_class, len(X_class))
            indices = np.random.choice(len(X_class), n_prototypes,
replace=False)
            for idx in indices:
                self.prototypes.append(X_class[idx].copy())
                self.prototype_labels.append(class_label)

        self.prototypes = np.array(self.prototypes)
        self.prototype_labels = np.array(self.prototype_labels)

        # Навчання
        for epoch in range(self.max_epochs):
```

```

# Перемішуємо дані
indices = np.random.permutation(len(X))
epoch_errors = 0

for i in indices:
    x = X[i]
    true_label = y[i]

    # Знаходимо найближчий прототип
    distances = np.linalg.norm(self.prototypes - x, axis=1)
    winner_idx = np.argmin(distances)
    winner_label = self.prototype_labels[winner_idx]
    # Оновлення за правилом LVQ
    if winner_label == true_label:
        # Правильна класифікація → наближаємо прототип
        self.prototypes[winner_idx] += self.learning_rate * (
            x - self.prototypes[winner_idx]
        )
    else:
        # Помилкова класифікація → віддаляємо прототип
        self.prototypes[winner_idx] -= self.learning_rate * (
            x - self.prototypes[winner_idx]
        )

    # Зменшуємо швидкість навчання з часом
    self.learning_rate *= 0.99

return self

def predict(self, X):
    """
    Передбачення класів для нових зразків
    """
    predictions = []
    for x in X:
        distances = np.linalg.norm(self.prototypes - x, axis=1)
        winner_idx = np.argmin(distances)
        predictions.append(self.prototype_labels[winner_idx])
    return np.array(predictions)

def predict_proba(self, X):
    """
    Повертає ймовірність класу 1 (на основі відстаней)
    """
    proba = []
    for x in X:
        distances = np.linalg.norm(self.prototypes - x, axis=1)
        # Знаходимо найближчі прототипи кожного класу
        min_dist_class0 = min(
            [
                distances[i]
                for i in range(len(self.prototypes))
                if self.prototype_labels[i] == 0
            ],
            default=np.inf,
        )

```

```

        min_dist_class1 = min(
            [
                distances[i]
                for i in range(len(self.prototypes))
                if self.prototype_labels[i] == 1
            ],
            default=np.inf,
        )

        # Ймовірність класу 1 (обернено пропорційна відстані)
        if min_dist_class0 + min_dist_class1 > 0:
            prob = min_dist_class0 / (min_dist_class0 + min_dist_class1)
        else:
            prob = 0.5
        proba.append(prob)
    return np.array(proba)

# Генерація даних
N = 500

X = np.random.rand(N, 18)
y = np.random.randint(0, 2, N)

# Розділення на навчальну та тестову вибірки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.8, random_state=42
)

# Створення та навчання LVQ
lvq = LVQ(n_prototypes_per_class=5, learning_rate=0.1, max_epochs=30)
lvq.fit(X_train, y_train)

# Передбачення
y_train_pred = lvq.predict(X_train)
y_test_pred = lvq.predict(X_test)

# Точність
train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)

print("\n2. Результати LVQ:")
print(f"    Точність на навчальних даних: {train_acc * 100:.2f}%")
print(f"    Точність на тестових даних: {test_acc * 100:.2f}%")

# Інформація про прототипи
print("\n3. Прототипи:")
print(f"    Кількість прототипів: {len(lvq.prototypes)}")
for label in [0, 1]:
    n_protos = np.sum(lvq.prototype_labels == label)
    print(f"    Клас {label}: {n_protos} прототипів")

"""
500 зразків, 5 прототипів, 100 епох
- 80/20: навчальна 87%, тестова 46%
500 зразків, 2 прототипи, 100 епох
- 80/20: навчальна 78%, тестова 49.50%

```

```
500 зразків, 5 прототипів, 50 епох
- 80/20: навчальна 82%, тестова 50.25%
500 зразків, 5 прототипів, 30 епох
- 80/20: навчальна 84%, тестова 54%
500 зразків, 5 прототипів, 20 епох
- 80/20: навчальна 86%, тестова 49%
""
```

```
data = {"Train Accuracy": train_acc, "Test Accuracy": test_acc}
current_folder = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(current_folder, "lvq_output.json")
with open(file_path, encoding="utf-8", mode="w") as f:
    json.dump(data, f, indent=2)
```

Результат в консолі

2. Результати LVQ:
Точність на навчальних даних: 85.00%
Точність на тестових даних: 51.50%
3. Прототипи:
Кількість прототипів: 10
Клас 0: 5 прототипів
Клас 1: 5 прототипів

Результати у файлі

```
{
  "Train Accuracy": 0.84,
  "Test Accuracy": 0.4775
}
```

Порівняння результатів

Під час виконання було зроблено експерименти з різними вхідними даними. Розбиття на тестову та навчальну вибірки завжди 80 на 20. Деталі на таблиці 2.1 нижче:

Таблиця 2.1 — Порівняння вхідних даних

Кількість зразків	Кількість прототипів	Кількість епох	Точність навчальної вибірки, %	Точність тестової вибірки, %
500	5	100	87	46
500	2	100	78	49.50
500	5	50	82	50.25
500	5	30	84	54
500	5	20	86	49